NET (BMEVITMAC09): two mock midterms at the level of the 2026 slides

Written by us, not official papers. Each question is the twin of an exercise or worked example actually shown in the 2026 slides (same type, same number of steps, same size of numbers), with new numbers; course questions stay at the level of detail of the slides. One question per session, sessions 1 to 6 in order. Format as announced by the lecturer and reported by Elisha: 6 open questions, 30 points, pass mark 12/30. Calculations checked by `calc_sujets_slides_net.py`.

Exercises and worked examples in the 2026 slides

Slide, page	Example	Difficulty
hta1 p. 13	PCM: 8 kHz × 8 bits = 64 kbit/s	1 multiplication
hta1p p. 18, 19	Token and leaky bucket design: K = average, B = (max −avg)× burst	formula, no numbers
hta1p p. 20	Statistical multiplexing: $N = 500$, $p = 1\%$, mean 5, Chebyshev bound 1/9	3 steps, small numbers
hta1p p. 24 to 26	Same record in XML, JSON, YAML; “Possible exam question”	2 scalars and 1 list of strings
hta2p p. 4, 7, 8	Class table, hosts per prefix ($2^{32-n} - 2$), private ranges	table lookup
hta2p p. 11	Ex. 1: netmask to prefix (255.224.0.0 = /11), /19 to netmask, invalid mask	1 step each
hta2p p. 12	Ex. 2: which of 5 addresses share 152.66.163.9/18 (binary)	5 binary comparisons
hta2p p. 13	Ex. 3: 3-line table, 1 destination, longest prefix match	binary, 1 destination
hta2p p. 14	Ex. 4: 4 lines on one interface merged into a /9, one misaligned line	2 merges, 1 alignment check
hta3 p. 10, 12; hta3p p. 10 to 20	DORA diagram; URL parts; ifconfig, ip, route examples; ping, traceroute	recall, command syntax
hta4p p. 5, 6	Distance vector on a line of 6 nodes; count to infinity after one node fails	3 to 5 rounds
hta4p p. 8 to 14	Dijkstra from C on 7 nodes, 9 links, queue shown per round	6 rounds
hta5 p. 9	3-way handshake: SYN (Sc), SYN,ACK (Ss, Sc+1), ACK (Sc+1, Ss+1)	symbolic
hta5 p. 18, 19	NAT: client, gateway inside and outside addresses, server	address rewriting
hta5p p. 8, 9	cwnd 1, 2, 4, 8 then 9, 10 with segment numbers (1; 2, 3; 4 to 7…)	counting per round
hta5p p. 13	Fast retransmit: segments 20 to 26, 21 lost, ACKs 21, 21, 21, 21, 21, 26	1 sequence
hta5p p. 16	16-bit window, window scale shift up to 14 bits, 1 GB	1 step
hta6 p. 8; hta6p p. 7 to 9	HTTP methods and code classes; SMTP, POP3, IMAP ports and commands	recall

Compared with the 2018 midterm, these papers drop what the slides never compute: VLSM allocation, aggregation with non-contiguous traps, router interface plans with ARP traces, Reno over many rounds with a capping convention, bandwidth-delay products, fragmentation tables.

Paper 1

6 questions, one per session, 5 points each, pass mark 12/30.

1. Session 1: XML to JSON and YAML (5 points)

mirrors hta1p p. 24 to 26 (“Possible exam question”)

```
<Router>
  <id>7</id>
  <name>Edge Router</name>
  <interfaces>
    <ifname>eth0</ifname>
    <ifname>eth1</ifname>
    <ifname>wlan0</ifname>
  </interfaces>
</Router>
```

(a) Turn the given XML into JSON. (2 pts)
(b) Turn it into YAML. (2 pts)
(c) Why was YAML introduced next to JSON, and what is it mainly used for? (1 pt)

2. Session 2: netmasks and same subnet (5 points)

mirrors hta2p p. 11 (Example 1) and p. 12 (Example 2)

(a) Prefix length of netmask 255.255.192.0? Netmask of /20? Is 255.255.240.128 valid? (1.5 pts)

(b) Which addresses are on the same subnet as 130.88.201.7/19? Show the 3rd octet in binary. 1) 130.88.215.40 2) 130.88.190.3 3) 131.88.201.9 4) 130.88.224.5 5) 130.88.192.12 6) 130.88.207.300 (3.5 pts)

3. Session 3: DHCP and configuration commands (5 points)

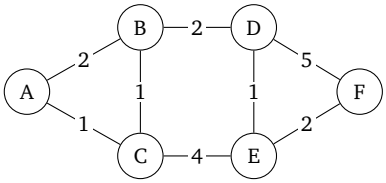
mirrors hta3 p. 10 (DORA diagram); hta3p p. 14 (iproute2 examples)

(a) Name the four DHCP messages in order. Which are broadcast, and why? Which transport and ports does DHCP use? (2.5 pts)

(b) With the `ip` command: give eth1 the address 172.16.5.20/24, bring it up, and add a route to 10.40.0.0/16 via 172.16.5.1. (2.5 pts)

4. Session 4: Dijkstra's algorithm (5 points)

mirrors hta4p p. 8 to 14 (LS example, run Dijkstra)



Our node is A.

(a) As on the slides, give for each round the node taken from the priority queue, the new path costs and the queue. (3.5 pts)

(b) Final cost and next hop from A for every node. (1.5 pts)

5. Session 5: slow start and congestion avoidance (5 points)

mirrors hta5p p. 8 and 9 (cwnd and segment numbers per round)

As on the slides: slow start doubles cwnd each round until cwnd reaches ssthresh; congestion avoidance adds 1 per round with ssthresh = cwnd; on a timeout ssthresh is halved and slow start restarts from cwnd = 1. Start: cwnd = 1, ssthresh = 4, segments numbered from 1.

(a) Give cwnd and the segment numbers sent in rounds 1 to 7. (3 pts)

(b) A timeout happens at the end of round 7. ssthresh, and cwnd in rounds 8 to 11? (2 pts)

6. Session 6: HTTP (5 points)

mirrors hta6 p. 8 to 11 (methods, response codes, versions, state)

(a) Which method: get a page; send a search term; get only the metadata of an object? (1.5 pts)

(b) What do the classes 2XX, 3XX, 4XX and 5XX mean? Give one code of each from the slides. (1.5 pts)

(c) What did HTTP/1.1, HTTP/2 and HTTP/3 each bring? How does a stateless protocol keep a shopping cart? (2 pts)

1. Session 1: PCM and statistical multiplexing (5 points)

mirrors hta1 p. 13 (PCM); hta1p p. 20 (statistical multiplexing)

(a) A wideband voice service keeps audio up to 8 kHz and quantizes each sample to 8 bits. Sampling rate and bitrate? (1.5 pts)

(b) $N = 300$ users, each using the network with probability $p = 2\%$. Average number of active users? Which distribution, and which approximation for large N and small p? Apply Chebyshev's inequality with $k = 3$ as on the slide: which L, and which bound on the blocking probability? (3.5 pts)

2. Session 2: longest prefix match and table compression (5 points)

mirrors hta2p p. 13 (Example 3) and p. 14 (Example 4)

(a) Table: 98.32.0.0/11 eth1; 98.48.0.0/12 eth4; 98.56.0.0/13 eth2. Outgoing interface for 98.52.130.9? Show the 2nd octets in binary. (2 pts)

(b) All four lines go to eth3: 200.17.36.0/21; 200.17.48.0/21; 200.17.56.0/22; 200.17.60.0/22. Which lines can be merged, into what? Which line is incorrectly given? (3 pts)

3. Session 3: URL, DNS and traceroute (5 points)

mirrors hta3 p. 6, 12, 14, 15; hta3p p. 18 to 20

(a) Split `https://www.example.org/docs/net/intro.html` into scheme, location, path and file. Which part does DNS resolve? (1.5 pts)

(b) What are the A, AAAA and MX records? How does a client know its local DNS resolver? (1.5 pts)

(c) How does traceroute find the routers on the path? Which ICMP message does it use? (2 pts)

4. Session 4: distance vector and count to infinity (5 points)

mirrors hta4p p. 5 (DV example) and p. 6 (count to infinity)

Five routers on a line: P, Q, R, S, T (hop count). Initially nothing is known.

(a) After how many rounds does P learn about T? What does R advertise once it knows all nodes within 2 hops? (1.5 pts)

(b) Q goes down. R detects it and marks P and Q unreachable. Then R receives S's update "P in 3 hops". Give the next three updates for destination P. (2.5 pts)

(c) How would split horizon and poisoned reverse prevent this? (1 pt)

5. Session 5: TCP handshake and fast retransmit (5 points)

mirrors hta5 p. 9 (3-way handshake); hta5p p. 13 (fast retransmit)

(a) The client picks Sc = 4100, the server Ss = 900. Give flags, Sequence and Acknowledgement of the three handshake segments. (2 pts)

(b) As on the slide, segments are numbered one by one. The sender sends 70 to 75; segment 72 is lost. Give the ACK the receiver returns for each segment, when Reno retransmits, and the ACK after 72 arrives. (3 pts)

6. Session 6: e-mail (5 points)

mirrors hta6p p. 4 and 7 to 9 (architecture, SMTP, POP3, IMAP)

(a) A message goes from Alice's mail client to Bob's mailbox and Bob reads it. Name the agents in order and the protocol at each step. How does the sending MTA find Bob's domain's MTA? (2.5 pts)

(b) Give the ports of SMTP (server to server and submission), POP3 and IMAP. (1 pt)

(c) Why is IMAP better than POP3 for reading mail on two devices? (1.5 pts)

Solutions

For each question, the filled boxes give the final answer to write on the paper; they are followed by the Method, the Formulas (with slide references; anything not in the 2026 slides is tagged), the Worked solution, a Check, and finally Marking and traps.

Paper 1: solutions

6 questions, one per session, 5 points each, pass mark 12/30.

1. Session 1: XML to JSON and YAML (5 points)

mirrors hta1p p. 24 to 26 ("Possible exam question")

```
<Router>
  <id>7</id>
  <name>Edge Router</name>
  <interfaces>
    <ifname>eth0</ifname>
    <ifname>eth1</ifname>
    <ifname>wlan0</ifname>
  </interfaces>
</Router>
```

(a) Turn the given XML into JSON. (2 pts)
(b) Turn it into YAML. (2 pts)
(c) Why was YAML introduced next to JSON, and what is it mainly used for? (1 pt)

Final answer

```
{
  "id": 7,
  "name": "Edge Router",
  "interfaces": [ "eth0", "eth1",
                  "wlan0" ]
}
```

```
id: 7
name: Edge Router
interfaces:
  - eth0
  - eth1
  - wlan0
```

(c) JSON is verbose with braces and brackets; YAML uses indentation like Python. Mainly for configuration files.

Method.

1. Drop the root element: its children become the keys of the top-level object.
2. An element that only contains text becomes *key: value*; in JSON, numbers stay without quotes, text goes in quotes.
3. An element whose children all repeat the same tag becomes one key (the container's name) whose value is an array; the repeated tag name disappears.
4. YAML: same tree, one *key: value* per line, two spaces of indentation per level, one "- " per list item, no quotes needed.
5. Count the values in the XML and in the result.

Formulas. None: the conversion follows the slide example where the same record is shown in XML, JSON (JER) and YAML (hta1p p. 24 to 26).

Worked solution.

XML	JSON	YAML
`<id>7</id>`	`"id": 7` (number, no quotes)	`id: 7`
`<name>Edge Router</name>`	`"name": "Edge Router"`	`name: Edge Router`
`<interfaces>` with 3 `<ifname>`	`"interfaces": ["eth0", "eth1", "wlan0"]`	`interfaces:` then `- eth0`, `- eth1`, `- wlan0`

(c) The slide gives the reason (JSON is too verbose with braces and brackets), the notation (indentation for hierarchy, like Python) and the main use (configuration files).

Check. The XML holds 5 text values (7, Edge Router, eth0, eth1, wlan0); the JSON and the YAML hold the same 5 values, in the same order.

Marking and traps. (a) object 1, array 0.5, 7 without quotes 0.5. (b) keys 1, hyphenated list 1. (c) 1. Traps: Keeping `ifname` inside the list. As on the slide, the root's content becomes the top-level object (an outer `"Router"` key is also consistent).

2. Session 2: netmasks and same subnet (5 points)

mirrors hta2p p. 11 (Example 1) and p. 12 (Example 2)

(a) Prefix length of netmask 255.255.192.0? Netmask of /20? Is 255.255.240.128 valid? (1.5 pts)

Final answer

192 = 11000000: /18. 255.255.240.0. Invalid: all 1s must come before all 0s.

(b) Which addresses are on the same subnet as 130.88.201.7/19? Show the 3rd octet in binary. 1) 130.88.215.40 2) 130.88.190.3 3) 131.88.201.9 4) 130.88.224.5 5) 130.88.192.12 6) 130.88.207.300 (3.5 pts)

Final answer

/19 = 8 + 8 + 3: compare the first 3 bits of the 3rd octet; 201 = 11001001. 215 = 11010111 **yes**; 190 = 10111110 no; 131.x: 1st octet differs, no; 224 = 11100000 no; 192 = 11000000 **yes**; 300 > 255: invalid.

Method.

1. Split the prefix length as $8 + 8 + \cdots + r$: as many 255 octets as full 8s, then one octet with r leading 1s, then 0s.
2. Convert each mask octet to binary (weights 128, 64, 32, 16, 8, 4, 2, 1). A valid mask is all 1s then all 0s.
3. Same subnet: check that every octet is ≤ 255 and that the leading octets are equal; write the deciding octet of each address in binary and compare its first r bits (equivalently, AND it with the mask octet and compare the results).

Formulas.

- Netmask of /n: n ones then $32 - n$ zeros (hta2p p. 6 and 11: 255.224.0.0 = /11, /19 = 255.255.224.0, 255.96.0.0 invalid).
- Same subnet $\iff$ same first n bits (hta2p p. 12).
- Network address = address AND mask [not in 2026 slides; it is the bit comparison of hta2p p. 12 written as an operation].

Worked solution. (a) 255.255.192.0: 255 = 11111111 (8 ones), 255 (8), 192 = 128 + 64 = 11000000 (2), 0 (0): 8 + 8 + 2 = /18. /20 = 8 + 8 + 4: 3rd octet 11110000 = 128 + 64 + 32 + 16 = 240: 255.255.240.0. 255.255.240.128: 240 = 11110000, 128 = 10000000: a 1 comes back after 0s, invalid.
(b) /19 = 8 + 8 + 3: compare the first 3 bits of the 3rd octet (mask octet 224 = 11100000). 201 = 128 + 64 + 8 + 1.

Address	3rd octet	Binary	AND 11100000	Result
130.88.201.7 (ours)	201 = 128 + 64 + 8 + 1	11001001	11000000 = 192	network 130.88.192.0
130.88.215.40	215 = 128 + 64 + 16 + 4 + 2 + 1	11010111	11000000 = 192	same
130.88.190.3	190 = 128 + 32 + 16 + 8 + 4 + 2	10111110	10100000 = 160	different
131.88.201.9	1st octet 131 = 10000011	vs 130 = 10000010		different
130.88.224.5	224 = 128 + 64 + 32	11100000	11100000 = 224	different
130.88.192.12	192 = 128 + 64	11000000	11000000 = 192	same
130.88.207.300	4th octet 300 > 255			invalid

Check. A /19 holds $2^{32-19} = 8192$ addresses = 32 values of the 3rd octet: 192 to 223. 215 and 192 are inside, 190 and 224 outside.

Marking and traps. (a) 0.5 each. (b) about 0.6 per address.

3. Session 3: DHCP and configuration commands (5 points)

mirrors hta3 p. 10 (DORA diagram); hta3p p. 14 (iproute2 examples)

(a) Name the four DHCP messages in order. Which are broadcast, and why? Which transport and ports does DHCP use? (2.5 pts)

Final answer

Discover, Offer, Request, Acknowledgement (DORA). Discover is broadcast (the client has no address and does not know the server); Request is also broadcast, to notify all DHCP servers. UDP, server port 67, client port 68.

(b) With the `ip` command: give eth1 the address 172.16.5.20/24, bring it up, and add a route to 10.40.0.0/16 via 172.16.5.1. (2.5 pts)

Final answer

`ip addr add 172.16.5.20/24 dev eth1;  ip link set dev eth1 up;  ip route add 10.40.0.0/16 dev eth1 via 172.16.5.1.`

Method.

1. DHCP: follow the client. It has no address, so it broadcasts Discover; servers answer with an Offer; the client broadcasts a Request for the offer it chose; the chosen server confirms with an Acknowledgement.
2. Commands: one `ip` command per action, in the order address, link up, route.

Formulas. None. Facts used: DHCP runs over UDP, server port 67, client port 68 (hta3 p. 8; hta5 p. 4); DORA and the broadcast Request (hta3 p. 10); command forms `ip addr add <addr/len> dev <if>`, `ip link set dev <if> up`, `ip route add <net/len> dev <if> via <gw>` (hta3p p. 14).

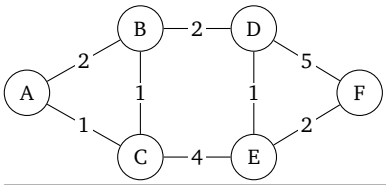
Worked solution. (a) 1. Discover, link layer and IP broadcast. 2. Offer (several servers may be active). 3. Request, broadcast to notify all DHCP servers. 4. Acknowledgement. (b) address 172.16.5.20/24 on eth1: `ip addr add 172.16.5.20/24 dev eth1`; up: `ip link set dev eth1 up`; route: `ip route add 10.40.0.0/16 dev eth1 via 172.16.5.1`.

Check. The gateway 172.16.5.1 shares the first 24 bits (172.16.5) with 172.16.5.20/24, so it is directly reachable on eth1, as a next hop must be (hta2 p. 6).

Marking and traps. (a) order 1, broadcast 1, ports 0.5. (b) about 0.8 per command; the older `ifconfig` and `route` forms earn partial credit.

4. Session 4: Dijkstra's algorithm (5 points)

mirrors hta4p p. 8 to 14 (LS example, run Dijkstra)



Our node is A.

(a) As on the slides, give for each round the node taken from the priority queue, the new path costs and the queue. (3.5 pts)

(b) Final cost and next hop from A for every node. (1.5 pts)

Final answer

Round	Taken	New paths; queue after the round
1	A (0)	A-B 2, A-C 1; queue C 1, B 2
2	C (1)	A-C-B 2 (not better), A-C-E 5; queue B 2, E 5
3	B (2)	A-B-D 4; queue D 4, E 5
4	D (4)	A-B-D-E 5 (not better), A-B-D-F 9; queue E 5, F 9
5	E (5)	A-C-E-F 7 (better); queue F 7
6	F (7)	done

(b) B 2 via B; C 1 via C; D 4 via B; E 5 via C; F 7 via C.

Method.

1. $N' = \{\text{source}\}$; $D(v)$ = link cost for neighbours, ∞ otherwise; $p(v)$ = source for neighbours.
2. Each round: take the node $u \notin N'$ with the smallest $D(u)$ from the priority queue and add it to N'.
3. For every neighbour $v \notin N'$ of u: if $D(u) + c(u,v) < D(v)$, set $D(v) = D(u) + c(u,v)$ and $p(v) = u$; otherwise keep the old values.
4. Stop when every node is in N'. Next hop: follow $p(\cdot)$ back to the source.

Formulas. Dijkstra with a priority queue (hta4p p. 3): `d = u.dist + dist(u, v); if d < v.dist: v.dist = d, v.prev = u`. Here $D(v) =$ `v.dist`, $p(v) =$ `v.prev`, $c(u,v) =$ `dist(u, v)`.

Worked solution. Each cell is $D(v)$, $p(v)$; a dash means v is already in N'.

Round	N'	B	C	D	E	F
1	A	2, A	1, A	∞	∞	∞
2	A, C	$1+1=2$, not <2: 2, A		∞	$1+4=5$: 5, C	∞
3	A, C, B			$2+2=4$: 4, B	5, C	∞
4	A, C, B, D				$4+1=5$, not <5: 5, C	$4+5=9$: 9, D
5	A, C, B, D, E					$5+2=7<9$: 7, E
6	all					7, E

Next hops: B (p = A) via B; C via C; D: $p(D)$ = B, $p(B)$ = A, via B; E: $p(E)$ = C, via C; F: $p(F)$ = E, $p(E)$ = C, via C.

Check. Every final $D(v)$ is the smallest $D(u)+c(u,v)$ over its neighbours: $F = \min(4+5, 5+2) = 7$; $E = \min(1+4, 4+1, 7+2) = 5$; $D = \min(2+2, 5+1, 7+5) = 4$; $B = \min(0+2, 1+1, 4+2) = 2$.

Marking and traps. (a) about 0.6 per round. (b) 1.5. Traps: Keeping F at 9.

5. Session 5: slow start and congestion avoidance (5 points)

mirrors hta5p p. 8 and 9 (cwnd and segment numbers per round)

As on the slides: slow start doubles cwnd each round until cwnd reaches ssthresh; congestion avoidance adds 1 per round with ssthresh = cwnd; on a timeout ssthresh is halved and slow start restarts from cwnd = 1. Start: cwnd = 1, ssthresh = 4, segments numbered from 1.

(a) Give cwnd and the segment numbers sent in rounds 1 to 7. (3 pts)

> Final answer
> cwnd 1, 2, 4, 5, 6, 7, 8; segments 1; 2, 3; 4 to 7; 8 to 12; 13 to 18; 19 to 25; 26 to 33.

(b) A timeout happens at the end of round 7. ssthresh, and cwnd in rounds 8 to 11? (2 pts)

> Final answer
> ssthresh = cwnd = 8 before, halved to **4**; cwnd 1, 2, 4, 5.

Method.

1. One line per round. The round sends cwnd segments, numbered from the next unsent one.
2. If cwnd < ssthresh (slow start): next cwnd = 2× cwnd. Once cwnd reaches ssthresh: congestion avoidance, next cwnd = cwnd +1 and ssthresh = cwnd.
3. Timeout: ssthresh is halved (in congestion avoidance ssthresh = cwnd, so it becomes cwnd/2), cwnd = 1, slow start again.

Formulas.

- Slow start: start with cwnd = 1, "after each successful Ack cwnd∗=2", until cwnd reaches ssthresh (hta5p p. 8).
- Congestion avoidance: cwnd + = 1, ssthresh = cwnd (hta5p p. 9).
- Timeout: ssthresh/=2, back to slow start (hta5p p. 8 and 9).
- First segment of a round = 1+ sum of the previous cwnd values; last = first + cwnd −1 [not in 2026 slides as a formula; the slides show the numbering 1; 2, 3; 4 to 7; 8 to 15].

Worked solution.

Round	cwnd, phase	First	Last	Next round
1	1, SS (1 < 4)	1	1	$2 \times 1 = 2$
2	2, SS (2 < 4)	2	$2 + 2 - 1 = 3$	$2 \times 2 = 4$
3	4, reaches ssthresh 4	4	$4 + 4 - 1 = 7$	CA: $4 + 1 = 5$, ssthresh 5
4	5, CA	8	$8 + 5 - 1 = 12$	6, ssthresh 6
5	6, CA	13	$13 + 6 - 1 = 18$	7, ssthresh 7
6	7, CA	19	$19 + 7 - 1 = 25$	8, ssthresh 8
7	8, CA	26	$26 + 8 - 1 = 33$	timeout: ssthresh = 8/2 = 4, cwnd = 1
8	1, SS			$2 \times 1 = 2$
9	2, SS			$2 \times 2 = 4$
10	4, reaches ssthresh 4			CA: 5
11	5, CA			

Check. $1 + 2 + 4 + 5 + 6 + 7 + 8 = 33$, the last segment number of round 7.

Marking and traps. (a) cwnd 1.5, segments 1.5. (b) 1 and 1. Traps: Doubling 4 to 8 in round 4.

6. Session 6: HTTP (5 points)

mirrors hta6 p. 8 to 11 (methods, response codes, versions, state)

(a) Which method: get a page; send a search term; get only the metadata of an object? (1.5 pts)

> Final answer
> GET; POST; HEAD.

(b) What do the classes 2XX, 3XX, 4XX and 5XX mean? Give one code of each from the slides. (1.5 pts)

> Final answer
> Success (200 OK); redirection (301 Moved Permanently); client error (403, 404 Not Found); server error (500, 502, 504).

(c) What did HTTP/1.1, HTTP/2 and HTTP/3 each bring? How does a stateless protocol keep a shopping cart? (2 pts)

> Final answer
> 1.1: reuse of the TCP connection for several objects. 2: header compression, request multiplexing. 3: QUIC instead of TCP. Cookies: the response sets a key-value pair, the browser sends it back with later requests to the same site.

Method.

1. Methods: GET retrieves the object of the URL, POST sends data in the request, HEAD retrieves only metadata.
2. A response code is classified by its first digit.
3. Versions: recall the one new feature the slides give for each.

Formulas. None. Facts: methods and code classes 1XX to 5XX (hta6 p. 8); versions (hta6 p. 10); cookies (hta6 p. 11).

Worked solution. (a) Get a page: GET. Search term: data put in the request, POST. Metadata only: HEAD. (b) First digit 2: success (200 OK); 3: redirection (301 Moved Permanently); 4: client error (403 Forbidden, 404 Not Found); 5: server error (500 Internal Server Error, 502 Bad Gateway, 504 Gateway Timeout). (c) HTTP/1.1 (RFC 2068): reuse of the TCP connection for more objects of the same server. HTTP/2 (SPDY): header compression, request multiplexing. HTTP/3: QUIC instead of TCP. HTTP is stateless; a cookie is a key-value pair set by a response and sent back by the user agent with later requests to the same site.

Check. Every code quoted starts with the digit of its class; only POST carries data in the request.

Marking and traps. (a) 0.5 each. (b) 0.75 for the classes, 0.75 for the examples. (c) 0.5 per version, 0.5 cookies. Traps: classifying 404 as a server error; saying that HTTP/1.1 introduced QUIC.

Paper 2: solutions

6 questions, one per session, 5 points each, pass mark 12/30.

1. Session 1: PCM and statistical multiplexing (5 points)

mirrors hta1 p. 13 (PCM); hta1p p. 20 (statistical multiplexing)

(a) A wideband voice service keeps audio up to 8 kHz and quantizes each sample to 8 bits. Sampling rate and bitrate? (1.5 pts)

> Final answer
> Nyquist: twice the maximum frequency, 16 kHz; $16{,}000 \times 8 =$ **128 kbit/s**.

(b) $N = 300$ users, each using the network with probability $p = 2\%$. Average number of active users? Which distribution, and which approximation for large N and small p? Apply Chebyshev's inequality with $k = 3$ as on the slide: which L, and which bound on the blocking probability? (3.5 pts)

Final answer

$Np = \mathbf{6}$. Binomial, approximated by Poisson with $\lambda = Np = 6$. As written on the slide ($\sigma = \lambda$): $L = 6 + 3 \times 6 = \mathbf{24}$, $\Pr(|X - 6| \geq 18) \leq 1/9$. With the true standard deviation $\sigma = \sqrt{6} \approx 2.45$: $L = 6 + 7.35 = 13.35$, so 14 links, same bound 1/9. Chebyshev does not depend on the distribution, so it overestimates.

Method.

1. PCM: sampling rate = twice the maximum frequency (Nyquist); bitrate = samples per second × bits per sample.
2. Statistical multiplexing: mean number of active users Np; the count is binomial (N trials, probability p), approximated by Poisson with $\lambda = Np$ for large N and small p.
3. Chebyshev with k: take $L = \mu + k\sigma$ links; then $\Pr(X > L) \leq \Pr(|X - \mu| \geq k\sigma) \leq 1/k^2$. Round L up to an integer.

Formulas.

- $f_s = 2f_{\max}$; bitrate = $f_s\times$ bits per sample; 8 kHz × 8 bits = 64 kbit/s (hta1 p. 13).
- Average Np; Binomial $\mu = Np$, "$\sigma = Np(1-p)$"; Poisson $\lambda = Np$, $\mu = \lambda$, "$\sigma = \lambda$"; blocking $\Pr(X > L) = 1-\text{CDF}(X, L)$; Chebyshev $\Pr(|X-\mu| \geq k\sigma) \leq 1/k^2$, independent of the distribution, so it overestimates (hta1p p. 20).
- The slide's "σ" values are in fact the variances; the true standard deviation is $\sqrt{Np(1-p)}$, or $\sqrt{\lambda}$ for Poisson [not in 2026 slides].

Worked solution. (a) $f_s = 2 \times 8$ kHz = 16 kHz = 16,000 samples/s; bitrate = 16,000 × 8 = 128,000 bit/s = 128 kbit/s.

(b) Mean: $Np = 300 \times 0.02 = 6$. Binomial variance $Np(1-p) = 6 \times 0.98 = 5.88$, close to $\lambda = 6$: the Poisson approximation fits. Slide convention ($\sigma = \lambda = 6$): $k\sigma = 3 \times 6 = 18$, $L = 6 + 18 = 24$, $\Pr(|X - 6| \geq 18) \leq 1/3^2 = 1/9 \approx 0.111$. With $\sigma = \sqrt{6} \approx 2.449$: $3\sigma \approx 7.35$, $L = 6 + 7.35 = 13.35$, rounded up to 14 links, same bound 1/9.

Check. The slide example has $\lambda = 5$ and $L = 5 + 3 \times 5 = 20 = 4\lambda$; here $L = 24 = 4 \times 6$, the same pattern. $128 = 2 \times 64$: twice the bandwidth, twice the telephone bitrate.

Marking and traps. (a) 0.75 each. (b) mean 1, distributions 1, L and bound 1.5; either σ convention accepted if stated.

2. Session 2: longest prefix match and table compression (5 points)

mirrors hta2p p. 13 (Example 3) and p. 14 (Example 4)

(a) Table: 98.32.0.0/11 eth1; 98.48.0.0/12 eth4; 98.56.0.0/13 eth2. Outgoing interface for 98.52.130.9? Show the 2nd octets in binary. (2 pts)

Final answer

32 = `00100000` (/11 fixes 3 bits), 48 = `00110000` (4 bits), 56 = `00111000` (5 bits); 52 = `00110100` matches lines 1 and 2, not 3: longest wins, **eth4**.

(b) All four lines go to eth3: 200.17.36.0/21; 200.17.48.0/21; 200.17.56.0/22; 200.17.60.0/22. Which lines can be merged, into what? Which line is incorrectly given? (3 pts)

Final answer

56/22 (56 to 59) + 60/22 (60 to 63) = 56/21 (only the last prefix bit differs); + 48/21 (48 to 55) = **200.17.48.0/20** (48 to 63). Line 1: in a /21 the 3rd octet is a multiple of 8, 36 is not; it really is 200.17.32.0/21 (32 to 39), and 40 to 47 are missing, so it cannot be merged.

Method.

1. Longest prefix match: write the deciding octet of every prefix and of the destination in binary; a line matches when the destination has the same first n bits; among the matching lines the longest prefix wins.
2. Compression: write each line's range; check that each network address is aligned (its host bits are 0); merge two lines with the same next hop when only the last bit of their prefixes differs (same size, contiguous); repeat with the result.

Formulas.

- "Longest prefix match wins" (hta2p p. 13).
- Merge rule: "merge two adjacent /18 lines into a /17 if first 17 bits are the same, and the next hop is the same" (hta2p p. 6); "we can merge if only the last bit of the prefix differs" (hta2p p. 14).
- A /n holds 2^{32-n} addresses (hta2p p. 7): in the 3rd octet a /22 spans 4 values, a /21 spans 8, a /20 spans 16.

Worked solution. (a) /11 = 8 + 3, /12 = 8 + 4, /13 = 8 + 5 bits of the 2nd octet.

Line	2nd octet	Prefix bits	52 = `00110100`
98.32.0.0/11 eth1	32 = `00100000`	`001`	`001` matches
98.48.0.0/12 eth4	48 = `00110000`	`0011`	`0011` matches
98.56.0.0/13 eth2	56 = `00111000`	`00111`	`00110` does not

Lines /11 and /12 match; the /12 is longer: eth4.

(b) /21 fixes 5 bits of the 3rd octet, /22 fixes 6.

Line	3rd octet	Prefix bits, host bits	Range
200.17.36.0/21	36 = `00100100`	`00100`, 100 ≠ 000: misaligned	really 32 to 39
200.17.48.0/21	48 = `00110000`	`00110`, 000	48 to 55
200.17.56.0/22	56 = `00111000`	`001110`, 00	56 to 59
200.17.60.0/22	60 = `00111100`	`001111`, 00	60 to 63

`001110` and `001111` differ only in the last bit: `00111` /21 = 200.17.56.0/21 (56 to 63). `00111` and `00110` (48/21) differ only in the last bit: `0011` /20 = 200.17.48.0/20 (48 to 63). The first line, really 32/21 (`00100`), would need 40/21 (`00101`) to form 32/20; 40 to 47 are not in the table, and 32/21 and 48/20 have different sizes: no merge. Result: 200.17.36.0/21 (really 32/21) and 200.17.48.0/20, both eth3.

Check. 3rd octet values covered before: 8 + 8 + 4 + 4 = 24; after: 8 + 16 = 24.

Marking and traps. (a) binary 1, result 1. (b) first merge 1, /20 1, misaligned line 1.

3. Session 3: URL, DNS and traceroute (5 points)

mirrors hta3 p. 6, 12, 14, 15; hta3p p. 18 to 20

(a) Split `https://www.example.org/docs/net/intro.html` into scheme, location, path and file. Which part does DNS resolve? (1.5 pts)

Final answer

Scheme `https`; location `www.example.org` (resolved to an IP address by DNS); path `/docs/net`; file `intro.html`.

(b) What are the A, AAAA and MX records? How does a client know its local DNS resolver? (1.5 pts)

Final answer

IPv4 address; IPv6 address; e-mail server. Typically from DHCP.

(c) How does traceroute find the routers on the path? Which ICMP message does it use? (2 pts)

Final answer

It sends packets with TTL = 1, 2, 3 ... ; the router where the TTL expires sends back ICMP Time Exceeded (type 11), which reveals it. Probes: UDP to a random port by default, ICMP Echo Request with `-I`, TCP SYN with `-T`.

Method.

1. URL: split at "`://`" (scheme), then at the first "`/`" (location), then at the last "`/`" (path, file).
2. DNS: the location is resolved to an IP address; the client asks the local resolver it knows from DHCP, for the record type it needs.
3. Traceroute: probes with TTL = 1, 2, 3 ... ; each router decrements the TTL; the router where it reaches 0 answers with ICMP Time Exceeded.

Formulas. None. Facts: `scheme://location/path/file` (hta3 p. 12); A, AAAA, MX records (hta3 p. 14); resolver typically from DHCP (hta3 p. 15); TTL decremented by each node, packet dropped at 0 (hta2 p. 8); Time Exceeded is ICMP type 11 (hta3 p. 4); traceroute probes: UDP random port by default, `-I` ICMP Echo Request, `-T` TCP SYN (hta3p p. 20).

Worked solution. (a) `https` | `www.example.org` | `/docs/net` | `intro.html`; DNS resolves `www.example.org`. (b) A: IPv4 address; AAAA: IPv6 address; MX: e-mail server. (c) TTL = 1: the first router decrements it to 0, drops the packet and returns Time Exceeded with its own address; TTL = 2: the second router does the same; and so on until the destination is reached.

Check. Location contains no "/"; path + "/" + file gives back `/docs/net/intro.html`.

Marking and traps. (a) 0.3 per part, 0.3 DNS. (b) 0.5 each. (c) TTL 1, Time Exceeded 1. Traps: calling `/docs/net/intro.html` the location; saying that routers answer with Echo Reply.

4. Session 4: distance vector and count to infinity (5 points)

mirrors hta4p p. 5 (DV example) and p. 6 (count to infinity)

Five routers on a line: P, Q, R, S, T (hop count). Initially nothing is known.

(a) After how many rounds does P learn about T? What does R advertise once it knows all nodes within 2 hops? (1.5 pts)

Final answer

T is 4 hops away: after **4 rounds**. R says [[P, 2], [Q, 1], [S, 1], [T, 2]], without next hops.

(b) Q goes down. R detects it and marks P and Q unreachable. Then R receives S's update "P in 3 hops". Give the next three updates for destination P. (2.5 pts)

Final answer

R does not know it is S's next hop: R sets P to **4 via S**. R says 4 to S; R is S's next hop towards P, so S updates to **5**. S says 5: R updates to **6**. And so on, counting to infinity.

(c) How would split horizon and poisoned reverse prevent this? (1 pt)

Final answer

Split horizon: S does not advertise P to R, its next hop. Poisoned reverse: S advertises P to R with an infinite cost, so R does not choose S as next hop.

Method.

1. Convergence: every round, each node learns the nodes one hop further; a node d hops away is learned after d rounds of exchanges.
2. An advertisement is a list of [destination, distance], without the next hop.
3. After a failure, a received "X in y hops" gives $y + 1$ via the sender; a router always accepts the change when it comes from its current next hop, even if the distance grows.

Formulas. "B says it can reach C in $y < x$ hops, so B is my next hop toward C with $y + 1$ hops" (hta4p p. 4); "if the diameter of the network is 10 hops this algorithm converges in 10 rounds" (hta4p p. 4); next hop stored but not part of the advertisement (hta4p p. 5); split horizon and poisoned reverse (hta4p p. 6).

Worked solution. (a) Distances from P: Q 1, R 2, S 3, T 4. After round k, P knows the nodes at most k hops away: round 1 {Q}, round 2 {Q, R}, round 3 {Q, R, S}, round 4 {Q, R, S, T}. R's advertisement: [[P, 2], [Q, 1], [S, 1], [T, 2]].

(b)

Step	Event	Distance to P
before	converged	R: 2 via Q; S: 3 via R; T: 4 via S
failure	Q down, R marks P and Q unreachable	R: ∞
1	S tells R "P in 3"	R: $3 + 1 = 4$ via S
2	R tells S "P in 4"; R is S's next hop	S: $4 + 1 = 5$ via R
3	S tells R "P in 5"	R: $5 + 1 = 6$ via S

(c) Split horizon: S, whose next hop towards P is R, does not advertise P to R, so step 1 never happens. Poisoned reverse: S advertises P to R with an infinite cost, so R never picks S.

Check. Each update adds exactly 1 to the advertised value: 3, 4, 5, 6.

Marking and traps. (a) rounds 0.75, advertisement 0.75. (b) about 0.8 per update. (c) 0.5 each. Traps: putting the next hop in the advertisement; making R refuse a worse route from its next hop.

5. Session 5: TCP handshake and fast retransmit (5 points)

mirrors hta5 p. 9 (3-way handshake); hta5p p. 13 (fast retransmit)

(a) The client picks Sc = 4100, the server Ss = 900. Give flags, Sequence and Acknowledgement of the three handshake segments. (2 pts)

Final answer
SYN (4100); SYN,ACK (900, 4101); ACK (4101, 901).

(b) As on the slide, segments are numbered one by one. The sender sends 70 to 75; segment 72 is lost. Give the ACK the receiver returns for each segment, when Reno retransmits, and the ACK after 72 arrives. (3 pts)

Final answer
70: 71; 71: 72; 73: 72; 74: 72; 75: 72. The third duplicate ACK 72 triggers the fast retransmit of 72; when it arrives, ACK **76** (73 to 75 were already received).

Method.
1. Handshake: SYN carries the client's number; SYN,ACK carries the server's number and acknowledges client +1; the final ACK carries client +1 and acknowledges server +1.
2. ACKs: the receiver always returns the next segment it wants. A missing segment makes the same ACK repeat; at the 3rd duplicate the sender retransmits it; when the gap is filled, the ACK jumps past everything already buffered.

Formulas. SYN (Sc); SYN,ACK (Ss, Sc+1); ACK (Sc+1, Ss+1) (hta5 p. 9). ACK = next segment the receiver wants, everything up to ACK−1 received (hta5 p. 11; hta5p p. 2). Fast retransmit after 3 duplicate ACKs (hta5p p. 13).

Worked solution. (a) SYN: seq 4100. SYN,ACK: seq 900, ack 4100 + 1 = 4101. ACK: seq 4101, ack 900 + 1 = 901.

(b)

Segment received	Received so far	ACK (next wanted)	Duplicates
70	70	71	
71	70, 71	72	
73	70, 71, 73	72	1st
74	70, 71, 73, 74	72	2nd
75	70, 71, 73, 74, 75	72	3rd: fast retransmit of 72
72 (retransmitted)	70 to 75	76	

Check. The final ACK 76 = 75 + 1, the first segment not sent yet.

Marking and traps. (a) about 0.7 per segment. (b) ACK list 1.5, moment 0.75, ACK 76 0.75.

6. Session 6: e-mail (5 points)

mirrors hta6p p. 4 and 7 to 9 (architecture, SMTP, POP3, IMAP)

(a) A message goes from Alice's mail client to Bob's mailbox and Bob reads it. Name the agents in order and the protocol at each step. How does the sending MTA find Bob's domain's MTA? (2.5 pts)

Final answer
MUA (submission) → MTA, SMTP → MTA of Bob's domain, SMTP → MDA (mailbox) → Bob's MUA, POP3 or IMAP. MX record in DNS.

(b) Give the ports of SMTP (server to server and submission), POP3 and IMAP. (1 pt)

Final answer
SMTP 25, submission 465 (formerly 587); POP3 110, 995 with TLS; IMAP 143, 993 with TLS.

(c) Why is IMAP better than POP3 for reading mail on two devices? (1.5 pts)

Final answer
IMAP queries the message database on the server (folders, browse without retrieving); POP3 only retrieves the inbox, optionally deleting it from the server.

Method.
1. Follow the message: submission by the MUA, relay between MTAs, storage by the MDA, retrieval by the receiver's MUA.
2. SMTP only pushes mail; retrieval is POP3 or IMAP.
3. The destination domain's MTA comes from its MX record in DNS.

Formulas. None. Facts: architecture MTA, MDA, MUA, MSA, MRA (hta6p p. 4); SMTP ports 25 and 465, formerly 587 (hta6p p. 7); POP3 110 and 995, IMAP 143 and 993 (hta6p p. 8 and 9); MX record (hta6p p. 7; hta3 p. 14).

Worked solution. (a) Alice's MUA submits with SMTP to her MTA; the MTA reads the MX record of Bob's domain and relays with SMTP to that domain's MTA; the MDA stores the message in Bob's mailbox; Bob's MUA retrieves it with POP3 or IMAP. (b) SMTP 25 between servers, 465 (formerly 587) for submission; POP3 110, 995 with TLS; IMAP 143, 993 with TLS. (c) IMAP works on the message database of the server (directories, headers and attachments separately, browsing without retrieval), so two devices see the same mailbox; POP3 only retrieves the inbox, optionally deleting it from the server.

Check. Each hop of (a) uses exactly one protocol and the last hop is a retrieval protocol, never SMTP.

Marking and traps. (a) agents 1, protocols 1, MX 0.5. (b) 0.25 each. (c) 1.5. Traps: retrieving mail with SMTP; giving IMAP port 110.